

Projeto de Iniciação Científica

Utilização de Dados de Vídeo
Aplicados à Visão Computacional
Visualizando Informações de Vídeo no Formato AV1

Prof. Ricardo Fabbri, Ph.D.



Grupo de Visualização
Instituto Politécnico – IPRJ
Universidade do Estado do Rio de Janeiro

Nova Friburgo, 31 de julho de 2026

Orientador

Prof. Ricardo Fabbri, Ph.D.

Aluno

Bernardo Brust Toledo Pereira

Projetos Relacionados

FAPERJ Jovem Cientista do Nosso Estado E25/2014 204167
FAPERJ APQ1 01/09335-8
FAPERJ E28/2014/204747
UERJ Prociência 2014–2017

1 Introdução e Justificativa

O presente plano de pesquisa refere-se a atividades propostas para a Iniciação Científica (IC), a serem realizadas no Laboratório de Visualização do Instituto Politécnico (IPRJ) da UERJ, vinculado ao Departamento de Modelagem Computacional (DMC). O objetivo central é expandir o escopo de pesquisa do laboratório, integrando o processamento e a análise de vídeo ao arsenal de ferramentas que atualmente foca primordialmente em imagens estáticas. Tal avanço será suportado pela construção de ferramentas de fácil integração para análise de vídeos, servindo como ponto de entrada para futuras investigações sobre os benefícios deste formato de mídia.

O vídeo consolidou-se como uma das formas predominantes de comunicação digital. Com o avanço de plataformas de *streaming*, a popularização das redes sociais e a maturidade da indústria de jogos, os casos de uso de conteúdo audiovisual são onipresentes. Nesse contexto, é fundamental que a visão computacional desfrute dos benefícios que esse formato oferece em relação ao uso de imagens estáticas.

Fundamentalmente, vídeos são sequências de imagens apresentadas em rápida sucessão [1]. Assim, técnicas consolidadas, como a reconstrução 3D a partir de imagens, podem ser adaptadas para vídeos, desde que as adequações necessárias sejam executadas. No entanto, é possível ir além: formatos modernos de compressão, como o AV1, utilizam técnicas avançadas de redução de redundância, como a *inter prediction*. Esta técnica gera vetores de movimento associados a blocos no vídeo para evitar repetições no arquivo final [2]. Tais vetores, originalmente projetados para compressão, possuem grande potencial para aplicações em visão computacional [3].

Para explorar esse potencial, são necessárias ferramentas eficientes e robustas que permitam a visualização de vetores de movimento, canais de cores e relações temporais entre quadros (*frames*). Tais ferramentas são úteis não apenas para análise direta, mas também para a integração em projetos futuros que explorem a visão computacional em vídeo [4].

Os *codecs* são softwares responsáveis por encodificar e decodificar formatos específicos de vídeo. Por exemplo, o *codec* de formato AV1 pode tanto produzir vídeos AV1 a partir de dados visuais (encodificação), como gerar os dados visuais a partir do arquivo de vídeo (decodificação). O diagrama abaixo demonstra o processo de encodificação:

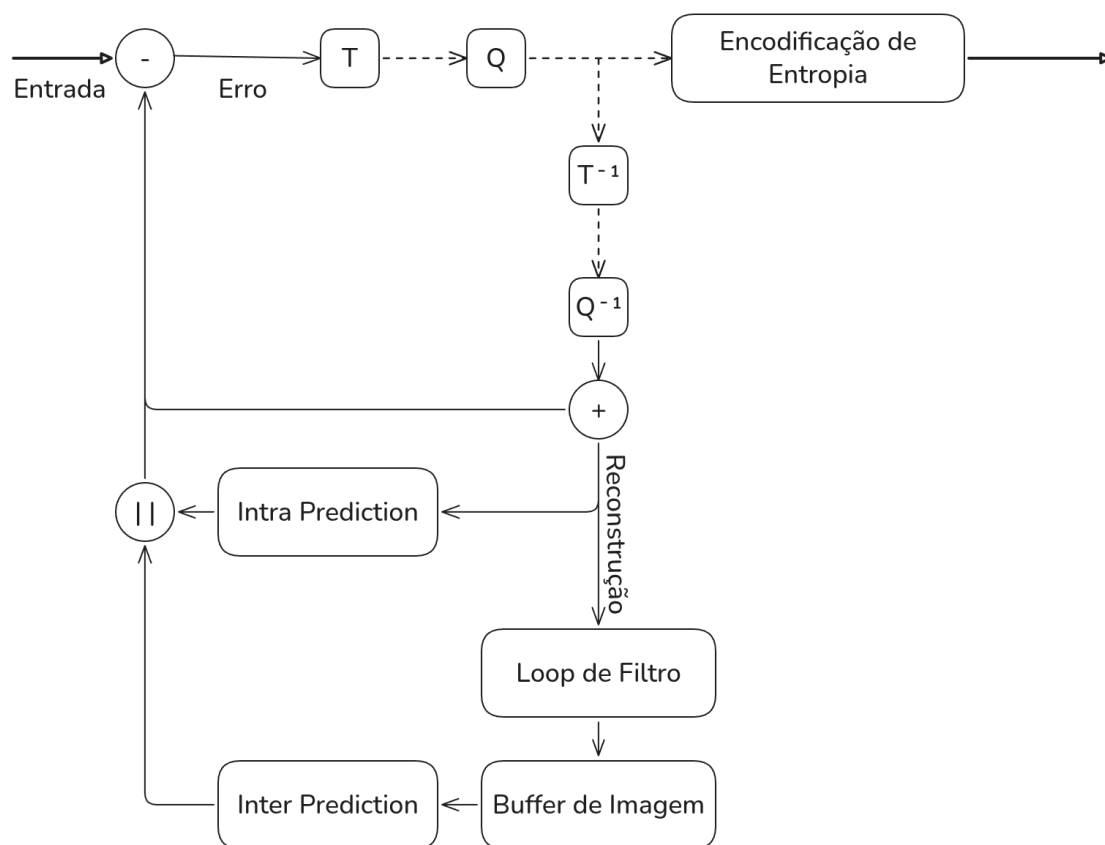


Figura 1: Diagrama demonstrando o processo de encodificação de vídeo.

As etapas funcionam da seguinte maneira:

- **Entrada:** O ponto de entrada da encodificação é onde o *codec* recebe os dados visuais puros que serão comprimidos. No ciclo do diagrama, estamos considerando um quadro (imagem) por vez como entrada.
- **T:** Esta é a primeira parte do processo onde é produzido erro. Uma transformada **T** é aplicada na imagem, fazendo com que ela perca detalhes irrelevantes. Esta parte é fundamental para reduzir o tamanho do arquivo e prepará-la para a próxima. Note que no diagrama a seta após a transformação é tracejada, ela ficará assim enquanto o quadro estiver transformado.
- **Q:** A quantização aplica uma função que reduz a precisão dos pixels da imagem. Esta parte é feita após a transformação por um motivo: ela é comumente uma função de arredondamento, enquanto a transformação comumente é uma transformada cossenoidal discreta. Logo, a perda de precisão não é significativa para o olho humano, mas anula varios dados, sendo assim um passo fundamental na compressão.

- **T inversa e Q inversa:** Há uma bifurcação nesta parte. Se um quadro já passou por todas as etapas no ciclo ele vai para a Encodificação de Entropia, caso contrário, ele desce no sentido do “+”, onde os dois primeiros passos desfazem a transformação e a quantização. É importante realizar estes passos, pois as etapas de reconstrução realizam mais compressões utilizando técnicas mais complexas que precisam de mais detalhes, mas vale também salientar que não há função inversa exata para os passos que foram realizados anteriormente, então ainda há um pouco de erro sendo propagado nas próximas etapas.
- **Loop de Filtro:** Esta etapa aplica filtros no quadro para alisar bordas e reduzir artefatos de compressão melhorando qualidade visual. Ela deve ser feita antes da *Inter Prediction* para melhorar a qualidade das compressões.
- **Buffer de Imagem:** Esta etapa armazena imagens anteriormente encodificadas para serem utilizadas posteriormente em *Inter Prediction*.
- **Inter Prediction:** Esta é uma das etapas mais importantes para este projeto. Nela, o *codec* utiliza os quadros anteriores e futuros para estimar o movimento de cada parte da imagem (cada parte é chamada de “block”, podendo ter tamanhos variados dependendo do nível de detalhe), isso gera um campo vetorial de movimento estimado de um quadro para o outro. Para o *codec* isso significa a possibilidade de armazenar vetores ao invés de uma nova imagem toda, sendo ótimo para compressão. Para a visão computacional, isso abre portas para diversas técnicas de compreensão espacial a partir dos campos vetoriais gerados.
- **Intra Prediction:** Assim como a etapa anterior, *Intra Prediction* comprime dados por previsão, mas neste caso, a *intra* não utiliza outros quadros para prever cada bloco, mas sim os blocos vizinhos de cada um para prever comportamentos como, por exemplo, iluminação (canal de luma). Esta parte também é relevante para visão computacional, pois o canal de iluminação pode ser utilizado para trabalhar o relevo de objetos de baixo detalhe.
- **Encodificação de Entropia:** A última parte de encodificação de cada quadro. Fundamentalmente ela substitui os códigos de valores comuns por valores mais curtos, e os menos comuns por mais longos. Em termos técnicos é um tipo de “*lossy compression*”.

2 Objetivos

Este projeto visa proporcionar ao aluno experiência com técnicas modernas de processamento de vídeo digital, decodificação de alta performance utilizando ferramentas como o *FFmpeg* [5], e análise de dados em visão computacional. O foco central será o desenvolvimento de ferramentas de extração e visualização de dados de vídeos no formato AV1.

O formato AV1 é de particular interesse para este projeto: ele é um formato aberto e isento de royalties. Isso significa que o *codec* do formato tem seu código fonte disponível na internet de

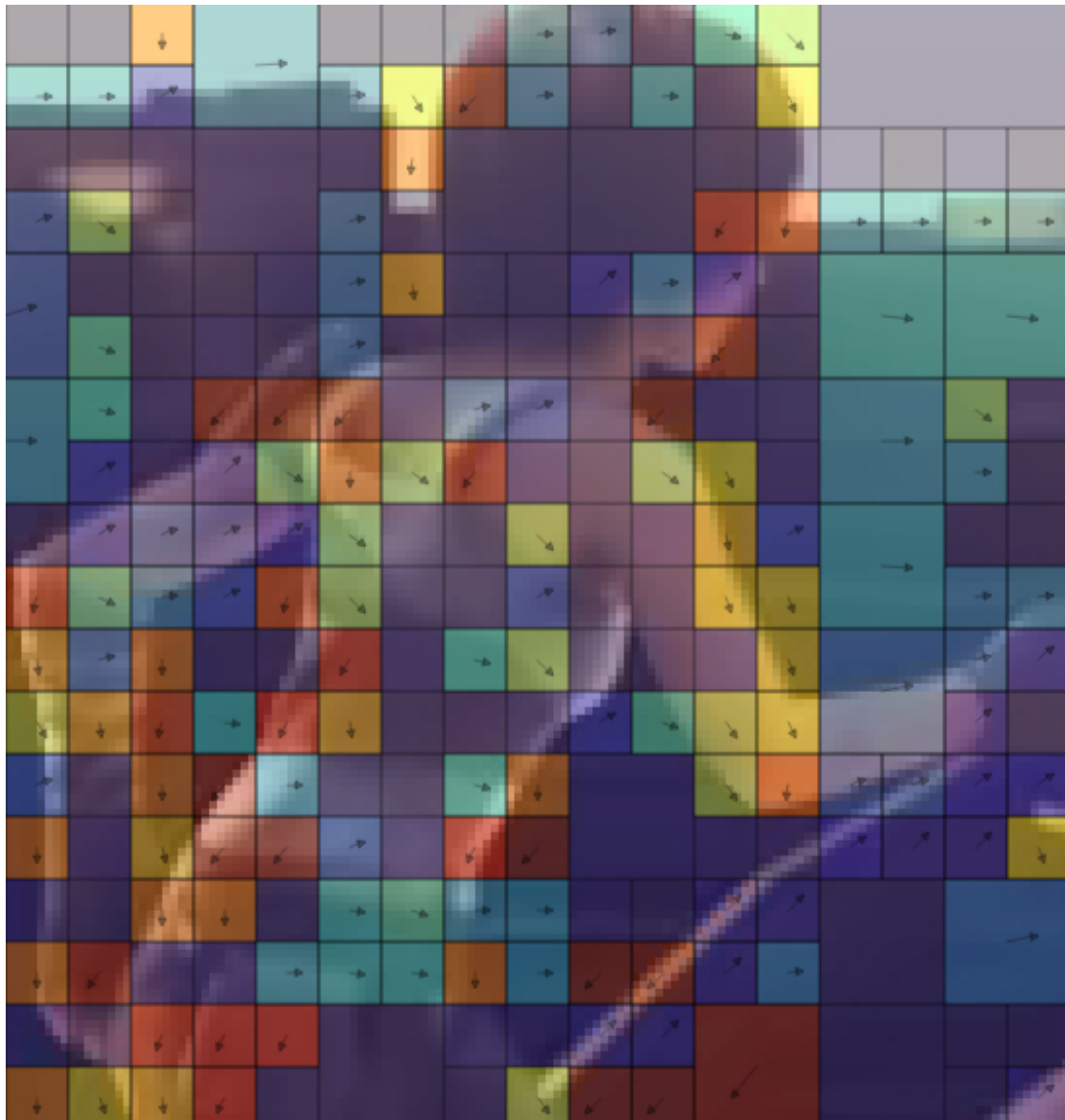
forma gratuita para uso e estudo. Ademais, estudos realizados pela Meta apontam que o formato é superior aos seus concorrentes em usos práticos [6].

Em relação ao hardware a ser utilizado, o uso do *codec* AV1 é um processo computacionalmente caro, podendo ser realizado tanto na CPU, de forma relativamente lenta, porém viável com ferramentas como *FFmpeg* [5], quanto na GPU, com decodificação acelerada em hardware pela API Vulkan [7], exigindo modelos de GPU modernos e caros. Para os propósitos deste trabalho, onde decodificação rápida não é uma necessidade, ferramenta será programada de modo a suportar a decodificação em CPU por ser mais acessível.

O sistema processará vídeos em formato AV1 para produzir representações gráficas dos dados presentes nos vídeos, como os vetores de movimento gerados durante a codificação, além de informações como canais de cores e sombreamento [2].

Os objetivos específicos incluem:

- Avaliar a precisão dos vetores de movimento gerados pelo *codec* para fins de visão computacional.
- Investigar a utilidade desses vetores em tarefas como a reconstrução 3D (*SfM*: *Structure from Motion*).
- Comparação com outras técnicas de *SfM* denso e *optical flow* [4].
- Analisar a viabilidade de determinar vetores de movimento em cenários com grandes deslocamentos nas posições da câmera.
- Explorar o uso de canais de croma para compreender a geometria de objetos com poucos detalhes.
- Avaliar a viabilidade de adaptar os métodos já utilizados no laboratório para o vídeo [8].
- Desenvolver uma ferramenta de código aberto que possibilite a visualização desses dados e sua integração em sistemas de pesquisa existentes.



Fonte: “Video Coding Basics - How is this so efficient? - Por Christian Feldmann”

Figura 2: Exemplo de visualização da *Intra Prediction* realizada no canal de luma de um vídeo encodificado em H.265. Vale notar que o formato AV1 realiza previsões melhores que as do H.265 [6].

A figura 2 apresenta um possível produto da ferramenta. Os vetores apresentados indicam uma diferença no comportamento da luz entre o quadro anterior e o atual. Tal campo vetorial é de suma importância para visualização aplicada à reconstrução 3D.

3 Metodologia

A execução do projeto será dividida nas seguintes etapas:

- **Construção da Camada de Plataforma:** Desenvolvimento da base do visualizador, incluindo a camada de plataforma, interface de usuário e interação.
- **Implementação do Visualizador de Vídeo:** Desenvolvimento das funcionalidades de inspeção, como reprodução quadro a quadro, câmera lenta e visualização lado a lado, utilizando a biblioteca *FFmpeg* [5] para decodificação na CPU.
- **Extração e Overlay de Dados:** Implementação da renderização de vetores de movimento e outros metadados diretamente sobre o vídeo.
- **Visualização de Canais Adicionais:** Inclusão de funcionalidades para visualizar canais de cores, croma e outros dados estruturais.
- **Investigação dos Efeitos em Bordas e Regiões com Baixa Textura:** Estudo do comportamento dos canais auxiliares em objetos de baixo detalhe. Como o uso de sombreamento para percepção de profundidade.
- **Otimização e Validação:** Análise de desempenho via *profiling* e garantia de qualidade por meio de testes automatizados.
- **Documentação e Publicação:** Disponibilização do código-fonte em repositório público, acompanhado de estudos de caso e guias de integração.

4 Cronograma

O cronograma de atividades está dividido em quatro etapas semestrais:

Atividades / Semestres	1	2	3	4
Construção da Camada de Plataforma	X			
Implementação do Visualizador de Vídeo	X	X		
Extração e Overlay de Dados		X	X	
Investigação dos Efeitos em Bordas e Regiões com Baixa Textura			X	X
Otimização e Validação			X	X
Documentação e Publicação		X	X	X

5 Conclusão

Este projeto propõe a exploração de técnicas modernas de visualização de dados aplicadas à análise de vídeo. Além de capacitar o aluno no estado da arte da visão computacional, o trabalho atende a uma demanda por ferramentas que auxiliem na transição do processamento de imagens estáticas

para o vídeo. O formato AV1, sendo aberto e de rápida adoção, representa um cenário ideal para esta investigação. Espera-se que a ferramenta desenvolvida contribua significativamente para a área, possibilitando aplicações futuras em reconstrução 3D em tempo real e análise geométrica baseada em dados de compressão.

Referências

- [1] K. Jack, *Video Demystified*. United States: Newnes, 4 ed., 2021. 1
- [2] Alliance for Open Media, *AV1 Bitstream and Decoding Process Specification*, 2019. Acessado em: 6 de junho de 2026. 1, 4
- [3] R. Szeliski, *Computer Vision: Algorithms and Applications*. United States: Springer, 2010. 1
- [4] J. Silva and M. Santos, “Leveraging AV1 motion vectors for fast and dense feature matching.” <https://arxiv.org/html/2510.17434v2>, 2025. 1, 4
- [5] FFmpeg, “FFmpeg video toolkit.” <https://ffmpeg.org>, 2026. 3, 4, 6
- [6] Meta, “AV1 beats x264 and libvpx-vp9 in practical use case.” <https://engineering.fb.com/2018/04/10/video-engineering/av1-beats-x264-and-libvpx-vp9-in-practical-use-case/>, 2018. 4, 5
- [7] Khronos Group, “Vulkan.” <https://www.vulkan.org/>, 2026. 4
- [8] “Trifocal relative pose from lines at points and its efficient solution,” 2019. 4